

Yes — this changes Bundle 11 in an important way.

The earlier Bundle 11 was too “Kubernetes CRD / repo-command wrapper” shaped. Your current Agentfield dev material shows a more specific direction:

```
Agentfield first = experiment-aware reasoner/controller POC
not yet = full Kubernetes-style CRD operator
not yet = only a shell wrapper around nca-art-grn
```

The diagram defines a **GRNExperiment CRD-like intent object**, status/results, a generic `BaseAgentfieldController`, a thin `ExperimentAwareController`, an Agent Registry, a Reasoner Invoker, and five GRN exploration agents: data profiling, dimensionality reduction, candidate regulator discovery, perturbation planning, and hypothesis ranking.

The developer’s POC confirms the concrete first implementation: start `af server`, set `OPENROUTER_API_KEY`, run `python grn_experiment.py`, then trigger `grn-experiment.run_experiment` through Agentfield’s `execute` endpoint. The workflow appears in the Agentfield UI as a DAG/call graph.

So Bundle 11 should be updated from “build a general experiment runtime wrapper” to:

```
Build the first Experiment-Aware Controller pattern inside Agentfield,
using the current single-file POC as the seed,
then harden it toward your NCA-ART-GRN research architecture.
```

Updated Bundle 11 — Agentfield experiment-aware controller foundation

Scrum-master view

Bundle objective: turn the current single-file Agentfield GRN POC into a structured, repo-backed Agentfield controller foundation.

This bundle should preserve the developer's working pattern:

```
Agentfield Agent node
-> run_experiment entrypoint
-> deterministic stage resolver
-> stage reasoners
-> accumulated status/results
-> Agentfield UI DAG
```

But adapt it to your actual platform:

```
not only scrna-seq regulator discovery
not only generic GRN analysis
not only LLM summaries

but eventually:
  DSL candidate mechanism
  PDE/ODE simulation evidence
  NCA evidence
  ART2 prototype evidence
  ARTMAP transition evidence
  perturbation-design evidence
  hypothesis ranking
  mechanism report status
```

The POC already implements `GRNExperimentSpec`, `StageResult`, `GRNExperimentStatus`, an `AGENT_REGISTRY`, stage reasoners, a deterministic `resolve_pipeline_stages` skill, and an async `run_experiment` reasoner that passes accumulated context forward.

Updated product decision

What Bundle 11 is now

Bundle 11 is:

Agentfield-native experiment controller foundation

It is **not yet**:

full Kubernetes CRD controller
full Paperclip adapter
full Runpod scheduler
full nca-art-grn execution campaign engine

The README makes the intent clear: the point is not full functionality yet; it is testing whether an **ExperimentAwareController** pattern feels natural inside Agentfield, where the controller reads structured experiment intent and decides a sequence of stages.

Updated platform relationship

config
prepares the Agentfield dev workspace, Python env, files, configs,
smoke scripts, and safe local checks

Agentfield
runs the experiment-aware controller node
exposes run_experiment through af server
shows the stage DAG in the UI

grn_experiment.py / Agentfield repo

contains the first controller POC:

- spec schema
- status schema
- agent registry
- reasoner functions
- stage resolver
- run_experiment orchestration

nca-art-grn repo later

provides real DSL/PDE/NCA/ART2/ARTMAP execution outputs

OpenClaw / PKM later

can reason over Agentfield status and reports

Paperclip later

sends experiment requests and reviews status/results

Updated user story

As the AI Engineer / Research Scientist,
I want the Agentfield GRN experiment-aware controller POC to become a structured Agentfield workspace,
so that I can submit a GRNExperiment intent, have Agentfield resolve and run the correct exploration stages, see the DAG in the UI, and gradually replace placeholder reasoners with real NCA-ART-GRN execution and mechanism evidence.

Updated concretizations / managed steps

```
prepare_agentfield_runtime_workspace  
prepare_agentfield_sdk_environment  
prepare_grn_experiment_poc_import  
prepare_grn_experiment_spec_schema  
prepare_grn_experiment_status_schema  
prepare_experiment_aware_controller_entrypoint  
prepare_agent_registry_runtime  
prepare_reasoner_invoker_runtime  
prepare_grn_exploration_reasoners  
prepare_grn_experiment_execute_fixtures  
prepare_agentfield_server_smoke_docs  
check_agentfield_runtime_ready  
run_agentfield_grn_poc_local_smoke
```

Optional next hardening steps:

```
prepare_grn_experiment_repo_split  
prepare_agentfield_nca_art_bridge  
prepare_agentfield_artifact_status_mapping  
prepare_agentfield_mechanism_report_status  
prepare_agentfield_runpod_target_stub
```

Those should come after the POC is stable.

What each step should do

`prepare_agentfield_runtime_workspace`

Type: CONFIG STEP

Owner: config prepares folders.

Target role: aiengineer .

Creates:

```
/workspace/repos/agentfield/  
  README.md  
  pyproject.toml  
  agentfield_grn/  
    __init__.py  
    schemas/  
    controllers/  
    registry/  
    reasoners/  
    invokers/  
    fixtures/  
    cli/  
  configs/  
    experiments/  
    reasoners/  
    agentfield/  
  smoke_tests/  
  runs/
```

What this does for the platform:

Creates a proper Agentfield development home instead of leaving the POC as a loose single file.

What this does for the research:

Gives your experiment-aware GRN orchestration its own platform repo while keeping scientific code in `nca-art-grn`.

How it links into the whole system:

```
config creates workspace  
aiengineer develops Agentfield controller there
```

Agentfield server loads/runs the controller

prepare_agentfield_sdk_environment

Type: CONFIG STEP

Owner: config prepares Python environment dependencies.

It should ensure the aiengineer environment can run the POC pattern:

```
agentfield SDK  
pydantic  
httpx or requests  
python-dotenv  
pyyaml  
rich
```

It should support environment variables used by the POC:

```
AGENTFIELD_URL  
AI_MODEL  
OPENROUTER_API_KEY
```

The developer's run instructions use `af server`, `OPENROUTER_API_KEY`, and `python grn_experiment.py`.

What this does for the platform:

Makes the Agentfield node runnable under the managed role/environment setup.

What this does for the research:

Lets you test experiment-aware orchestration without touching the GRN research repo yet.

How it links into the whole system:

```
config prepares aiengineer env  
Agentfield POC runs as aiengineer  
later Paperclip/Agentfield adapter can reuse the same endpoint
```

`prepare_grn_experiment_poc_import`

Type: CONFIG STEP

Owner: config imports/seeds POC into repo structure.

Input: current `grn_experiment.py` .

Current POC has:

```
GRNExperimentSpec  
StageResult  
GRNExperimentStatus  
Agent node setup  
AGENT_REGISTRY  
five reasoners  
resolve_pipeline_stages skill  
run_experiment reasoner  
app.run(auto_port=True)
```

The step should place it as:

```
/workspace/repos/agentfield/agentfield_grn/controllers/grn_experiment.py
```

or temporarily:

```
/workspace/repos/agentfield/poc/grn_experiment.py
```


What this does for the platform:

Preserves the developer's working implementation as the starting point.

What this does for the research:

Gives you a running controller before over-engineering the architecture.

How it links into the whole system:

```
uploaded POC -> managed Agentfield repo  
Agentfield server -> run_experiment  
UI -> workflow DAG
```

prepare_grn_experiment_spec_schema

Type: CONFIG STEP

Creates/validates AGENTFIELD CODE:

```
agentfield_grn/schemas/grn_experiment.py  
configs/experiments/grn_experiment.schema.yaml
```

Current POC schema fields:

```
name  
description  
dataset_ref  
organism  
method_flags
```

Keep those, but extend carefully toward your research:

```
experiment_id
name
description
dataset_ref
organism
method_flags
research_mode
candidate_id
candidate_batch_id
mechanism_hypothesis_id
config_ref
expected_outputs
```

Suggested `research_mode` values:

```
bioinformatics_grn_poc
nca_art_grn_smoke
mechanism_discovery
parameter_search_review
perturbation_design
```

What this does for the platform:

Turns the POC input into a stable experiment intent object.

What this does for the research:

Lets one Agentfield entrypoint eventually support both the current bioinformatics-style GRN POC and the actual NCA-ART-GRN mechanism-discovery workflow.

How it links into the whole system:

```
Paperclip later submits this schema
Agentfield validates it
```



controller resolves stages from it
next agent later receives config ref/candidate refs

prepare_grn_experiment_status_schema

Type: CONFIG STEP

Creates/validates AGENTFIELD CODE:

```
agentfield_grn/schemas/status.py  
configs/experiments/grn_experiment_status.schema.yaml
```

Current POC status has:

```
phase  
selected_agents  
stage_results  
final_summary
```

Extend to:

```
phase  
selected_agents  
stage_results  
execution_refs  
artifact_refs  
report_refs  
failure_reason  
final_summary  
human_review_required
```

Keep the POC's StageResult pattern:

```
stage
status
summary
data
```

What this does for the platform:

Makes the experiment self-contained and inspectable, matching your README intent that status/results should be visible on the experiment object rather than buried in logs.

What this does for the research:

Every stage result can later point to mechanism evidence: NCA summary, ART2 prototypes, ARTMAP transitions, perturbation plans, hypothesis rankings.

How it links into the whole system:

```
controller writes status
Agentfield UI displays call graph/results
Paperclip later maps this status into dashboard items
```

```
prepare_experiment_aware_controller_entrypoint
```

Type: CONFIG STEP

Creates/validates AGENTFIELD CODE:

```
agentfield_grn/controllers/experiment_aware_controller.py
```

This should preserve the POC's main pattern:

```
@app.reasoner()
async def run_experiment(...):
    resolve stages
    initialize status
    execute stages sequentially
    pass accumulated context forward
    return status
```

But split responsibilities:

```
controller = orchestration flow
schemas = Pydantic models
registry = stage-to-agent mapping
invoker = calls stage reasoners
reasoners = individual agents
```

What this does for the platform:

Creates the first experiment-aware controller specialization.

What this does for the research:

Lets your GRN exploration flow become explicit and inspectable: not one black-box prompt, but stages with status.

How it links into the whole system:

```
GRNExperimentSpec -> controller -> registry -> invoker -> reasoners -> status
```

prepare_agent_registry_runtime

Type: CONFIG STEP**Creates/validates AGENTFIELD CODE:**

```
agentfield_grn/registry/agent_registry.py  
configs/reasoners/grn_agent_registry.yaml
```

The POC registry maps:

```
data_profiling -> profile_data  
dimensionality_reduction -> reduce_dimensions  
candidate_regulators -> find_candidate_regulators  
perturbation_planning -> plan_perturbations  
hypothesis_ranking -> rank_hypotheses
```

Keep that, but prepare future NCA-ART-GRN mappings:

```
dsl_candidate_review -> review_dsl_candidate  
mechanism_hypothesis_review -> review_mechanism_hypothesis  
nca_art_evidence_review -> review_nca_art_evidence  
art2_prototype_review -> review_art2_prototypes  
artmap_transition_review -> review_artmap_transitions  
mechanism_report_review -> review_mechanism_report
```

What this does for the platform:

Keeps stage selection deterministic and inspectable.

What this does for the research:

Allows the controller to select the right expert reasoner for each experiment stage rather than pushing all meaning into one agent.

How it links into the whole system:

```
method_flags/research_mode -> stages  
stages -> reasoner functions  
selected_agents -> status.selected_agents
```

prepare_reasoner_invoker_runtime

Type: CONFIG STEP

Creates/validates AGENTFIELD CODE:

```
agentfield_grn/invokers/reasoner_invoker.py
```

The invoker should handle:

```
passing dataset_ref / organism / description  
passing prior_context  
collecting StageResult  
catching reasoner errors  
recording failed stage  
returning partial status
```

The POC currently does this directly in `run_experiment` ; this step extracts it into a clean helper.

What this does for the platform:

Keeps the controller thin, as intended in your diagram/README.

What this does for the research:

Allows real stage reasoners to fail independently without losing the whole experiment context.

How it links into the whole system:

```
controller decides stages
invoker calls reasoners
StageResult accumulates
status records completed/failed stage
```

`prepare_grn_exploration_reasoners`

Type: CONFIG STEP

Creates/validates AGENTFIELD CODE:

```
agentfield_grn/reasoners/data_profiling.py
agentfield_grn/reasoners/dimensionality_reduction.py
agentfield_grn/reasoners/candidate_regulators.py
agentfield_grn/reasoners/perturbation_planning.py
agentfield_grn/reasoners/hypothesis_ranking.py
```

Current five POC reasoners:

```
Data Profiling Agent
Dimensionality Reduction Agent
Candidate Regulator Agent
Perturbation Planning Agent
Hypothesis Ranking Agent
```

These match the five agents in the diagram.

But for your actual research, annotate them as **POC bioinformatics reasoners**, then add stubs for **NCA-ART-GRN reasoners**:


```
DSL Candidate Review Agent  
Mechanism Hypothesis Review Agent  
NCA-ART Evidence Review Agent  
ART2 Prototype Review Agent  
ARTMAP Transition Review Agent  
Perturbation Design Review Agent  
Mechanism Report Review Agent
```

What this does for the platform:

Separates reasoner modules and makes the DAG extensible.

What this does for the research:

Keeps the current developer POC working while preparing the real research stages you need.

How it links into the whole system:

```
current:  
  scrna-seq style GRN exploration reasoners  
  
next:  
  NCA-ART-GRN mechanism evidence reasoners  
  
later:  
  Agentfield can choose either flow from research_mode
```

```
prepare_grn_experiment_execute_fixtures
```

Type: CONFIG STEP

Creates smoke fixtures:

```
smoke_tests/fixtures/grn_discovery_human_cortex.json
smoke_tests/fixtures/nca_art_mechanism_smoke.json
```

Keep the developer's current trigger fixture:

```
{
  "input": {
    "name": "GRN Discovery in Human Cortex",
    "description": "Identify key transcription factor regulatory networks in human cortical development using s",
    "dataset_ref": "GSE123456_cortex_scrna",
    "organism": "human",
    "method_flags": ["pca", "correlation", "perturbation"]
  }
}
```

This exact shape is from the developer's trigger example.

Add your future fixture:

```
{
  "input": {
    "name": "NCA-ART-GRN Mechanism Smoke",
    "description": "Evaluate one DSL-defined 5-node GRN mechanism using simulator, NCA, ART2, ARTMAP and pertur",
    "dataset_ref": "/workspace/runs/nca-art-grn/smoke/latest",
    "organism": "synthetic",
    "method_flags": ["dsl", "nca", "art2", "artmap", "perturbation", "hypothesis_ranking"],
    "research_mode": "nca_art_grn_smoke"
  }
}
```

What this does for the platform:

Provides reproducible execute payloads.

What this does for the research:

Shows the migration path from generic GRN exploration to your actual NCA-ART-GRN mechanism workflow.

How it links into the whole system:

```
fixtures -> curl/execute endpoint  
Agentfield server -> run_experiment  
UI -> workflow DAG
```

prepare_agentfield_server_smoke_docs

Type: CONFIG STEP

Creates docs/scripts:

```
smoke_tests/run_agentfield_server.md  
smoke_tests/curl_grn_experiment.sh  
smoke_tests/curl_grn_experiment_async.sh
```

Must document the known flow:

```
af server  
export OPENROUTER_API_KEY=...  
python grn_experiment.py  
curl -X POST http://localhost:8080/api/v1/execute/grn-experiment.run_experiment ...
```

The developer also notes that Agentfield has an async execute endpoint that returns an `execution_id` if the pipeline exceeds HTTP timeout.

What this does for the platform:

Makes the POC reproducible.

What this does for the research:

Lets you demo and validate the controller pattern before connecting real experiment execution.

How it links into the whole system:

```
aiengineer follows smoke docs
Agentfield UI shows DAG
researchscientist reviews whether stage flow matches research intent
```

`check_agentfield_runtime_ready`

Type: CONFIG CHECK STEP

Runs: no model calls, no server start.

Should check:

```
Agentfield workspace exists
agentfield SDK importable
pydantic importable
grn_experiment module exists
GRNExperimentSpec importable
GRNExperimentStatus importable
run_experiment entrypoint present
fixtures exist
AGENTFIELD_URL configured or defaultable
```

AI_MODEL configured or defaultable
OPENROUTER_API_KEY present or clearly missing

What this does for the platform:

Validates readiness without spending tokens.

What this does for the research:

Avoids confusing missing keys or SDK issues with controller design failure.

How it links into the whole system:

operator/aiengineer runs check
fixes SDK/env/key/workspace issues
then runs actual POC smoke manually or via config

`run_agentfield_grn_poc_local_smoke`

Type: CONFIG SMOKE EXECUTION STEP

Runs: minimal local POC only.

Because the real Agentfield server may need interactive lifecycle, this step can have two modes:

dryrun:
 imports module
 validates fixture JSON
 validates stage resolution
 does not call model

live:
 assumes af server running
 assumes OPENROUTER_API_KEY set

calls execute endpoint
records execution response

Dryrun output:

```
/workspace/runs/agentfield/smoke/<timestamp>/  
  fixture.json  
  resolved_stages.json  
  selected_agents.json  
  dryrun_status.json
```

Live output:

```
/workspace/runs/agentfield/smoke/<timestamp>/  
  fixture.json  
  execute_response.json  
  execution_id.txt  
  status.json
```

What this does for the platform:

Proves the POC wiring works.

What this does for the research:

Confirms Agentfield can represent GRN experiment intent, choose stages, execute reasoners, and surface results in status/DAG form.

How it links into the whole system:

```
today:  
  run current developer POC  
  
next:  
  add NCA-ART-GRN reasoner stubs
```

```
later:
  call nca-art-grn repo outputs from Agentfield

later:
  Panerclin adapter submits the same experiment intent
```

Updated acceptance criteria

The following should become valid:

```
sudo config --target aiengineer bootstrap step prepare_agentfield_runtime_workspace
sudo config --target aiengineer bootstrap step prepare_agentfield_sdk_environment
sudo config --target aiengineer bootstrap step prepare_grn_experiment_poc_import
sudo config --target aiengineer bootstrap step prepare_grn_experiment_spec_schema
sudo config --target aiengineer bootstrap step prepare_grn_experiment_status_schema
sudo config --target aiengineer bootstrap step prepare_experiment_aware_controller_entrypoint
sudo config --target aiengineer bootstrap step prepare_agent_registry_runtime
sudo config --target aiengineer bootstrap step prepare_reasoner_invoker_runtime
sudo config --target aiengineer bootstrap step prepare_grn_exploration_reasoners
sudo config --target aiengineer bootstrap step prepare_grn_experiment_execute_fixtures
sudo config --target aiengineer bootstrap step prepare_agentfield_server_smoke_docs
sudo config --target aiengineer bootstrap step check_agentfield_runtime_ready
```

Explicit smoke:

```
sudo config --target aiengineer bootstrap step run_agentfield_grn_poc_local_smoke
```

Prepare/check steps must:

```
not launch Agentfield server
not call OpenRouter
not print OPENROUTER_API_KEY
not modify nca-art-grn research code
not require Paperclip
not require Runpod
not claim real GRN discovery from POC text outputs
```

Live smoke may call Agentfield/OpenRouter only if explicitly configured.

Updated proposed tests

Registry and syntax tests

```
bash --noprofile --norc -n /home/vmuser/.local/bin/config.sh
bash --noprofile --norc -n /home/vmuser/.local/lib/config-sh/installers.sh

config bootstrap steps | grep prepare_agentfield_sdk_environment
config bootstrap steps | grep prepare_grn_experiment_poc_import
config bootstrap steps | grep prepare_experiment_aware_controller_entrypoint
config bootstrap steps | grep prepare_agent_registry_runtime
config bootstrap steps | grep run_agentfield_grn_poc_local_smoke
```

Workspace tests

```
sudo config --target aiengineer bootstrap step prepare_agentfield_runtime_workspace

test -d /workspace/repos/agentfield/agentfield_grn/schemas
test -d /workspace/repos/agentfield/agentfield_grn/controllers
test -d /workspace/repos/agentfield/agentfield_grn/registry
```



```
test -d /workspace/repos/agentfield/agentfield_grn/reasoners
test -d /workspace/repos/agentfield/agentfield_grn/invokers
test -d /workspace/repos/agentfield/smoke_tests
```

POC import tests

```
sudo config --target aiengineer bootstrap step prepare_grn_experiment_poc_import

test -f /workspace/repos/agentfield/agentfield_grn/controllers/grn_experiment.py
grep "GRNExperimentSpec" /workspace/repos/agentfield/agentfield_grn/controllers/grn_experiment.py
grep "run_experiment" /workspace/repos/agentfield/agentfield_grn/controllers/grn_experiment.py
```

Schema tests

```
sudo config --target aiengineer bootstrap step prepare_grn_experiment_spec_schema
sudo config --target aiengineer bootstrap step prepare_grn_experiment_status_schema

test -f /workspace/repos/agentfield/configs/experiments/grn_experiment.schema.yaml
test -f /workspace/repos/agentfield/configs/experiments/grn_experiment_status.schema.yaml
```

The spec must include at least:

```
name
description
dataset_ref
organism
method_flags
```

The status must include at least:

```
phase  
selected_agents  
stage_results  
final_summary
```

Registry tests

```
sudo config --target aiengineer bootstrap step prepare_agent_registry_runtime  
  
test -f /workspace/repos/agentfield/configs/reasoners/grn_agent_registry.yaml  
grep "data_profiling" /workspace/repos/agentfield/configs/reasoners/grn_agent_registry.yaml  
grep "hypothesis_ranking" /workspace/repos/agentfield/configs/reasoners/grn_agent_registry.yaml
```

Fixture tests

```
sudo config --target aiengineer bootstrap step prepare_grn_experiment_execute_fixtures  
  
test -f /workspace/repos/agentfield/smoke_tests/fixtures/grn_discovery_human_cortex.json  
test -f /workspace/repos/agentfield/smoke_tests/fixtures/nca_art_mechanism_smoke.json
```

Readiness check

```
sudo config --target aiengineer bootstrap step check_agentfield_runtime_ready
```

Expected: readiness report only; no model call.

Dryrun smoke

```
sudo config --target aiengineer bootstrap step run_agentfield_grn_poc_local_smoke

find /workspace/runs/agentfield/smoke -name resolved_stages.json | tail -n 1
find /workspace/runs/agentfield/smoke -name selected_agents.json | tail -n 1
find /workspace/runs/agentfield/smoke -name dryrun_status.json | tail -n 1
```

Expected dryrun stage output for default flags:

```
data_profiling
dimensionality_reduction
candidate_regulators
perturbation_planning
hypothesis_ranking
```

This follows the POC's `resolve_pipeline_stages` logic: profiling always runs, dimensionality runs for PCA/UMAP/t-SNE flags, candidate regulators run for correlation/mutual-information/GRN flags, perturbation planning runs when `perturbation` is enabled, and hypothesis ranking always runs last.

Updated scientific guardrail

The current POC is valuable, but it is not yet your full research engine.

It should be labelled:

```
POC bioinformatics GRN exploration controller
```

not:

```
completed NCA-ART-GRN mechanism discovery platform
```

The next hardening stage should gradually replace or extend the five placeholder/bioinformatics reasoners with your real evidence chain:

```
DSL candidate review
PDE/ODE simulation evidence review
NCA rollout/agreement review
ART2 prototype review
ARTMAP transition review
perturbation design review
mechanism hypothesis ranking
```

Updated Bundle 11 summary

Bundle 11 now becomes:

Take the working Agentfield POC and make it a structured, managed, experiment-aware controller foundation.

The key update is:

Do not jump straight to a heavy CRD/operator abstraction.

First preserve the working Agentfield pattern:

```
Agent node
run_experiment entrypoint
deterministic stage resolver
agent registry
reasoner invoker
stage results
```

Then extend it toward your real platform:

NCA-ART-GRN research evidence
mechanism reports
artifact refs
Paperclip adapter
Agentfield campaigns

In one line:

Bundle 11 is the bridge from the developer's working Agentfield POC to your real experiment-aware GRN dis